

Themisra Licensing Schemas

Stefan Stefanović, Ligate Labs

2026-05-26

Themisra Licensing Schemas

Prompt + Content Licensing as Receipt-Layer Extensions

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-27

Status: v0.2 promotes the v0.1 outline to substantive content across all sections. §2 surveys prior art (NFT-as-content patterns, ASCAP/BMI, Story Protocol, EAS revocable attestations). §3 + §4 specify both licensing schemas with concrete fields and invariants. §5 specifies the cross-schema composition mechanism with cascade semantics. §6 specifies royalty distribution mechanics (25/35/30/10 burn/attestor/creator/builder split) with a worked creator-economy example in §A. §7 specifies adversarial considerations (license stripping, derivative-work edge cases, sublicensing bounds). Appendix B (formal schema definitions in CSC v0.2 §3.1 format) reserved for v0.3.

Contact: hello@ligate.io

Version history: v0.1 (2026-05-26, outline). v0.2 (2026-05-27, substantive content across all sections + worked creator-economy round trip in Appendix A).

Contents

Themisra Licensing Schemas	1
Prompt + Content Licensing as Receipt-Layer Extensions	1
Abstract	3
1. Introduction	3
1.1 Why licensing belongs as a schema, not as a chain-layer token primitive	3
1.2 Why now (or why v1.5)	3
1.3 The central question	4
1.4 Approach in brief	4
1.5 Contributions	5
1.6 Scope and non-goals	5
1.7 Document structure	5
2. Background and Related Work	5
2.1 NFT-as-content patterns	6
2.2 ASCAP and BMI: off-chain music licensing	6
2.3 Story Protocol and other IP-on-chain projects	6
2.4 EAS revocable attestations + license patterns	7
3. The <code>themisra.prompt-licensing/v1</code> Schema	7
3.1 Schema fields	7
3.2 Invariants	8
3.3 Version-bumping rules	9
3.4 Worked example	9
4. The <code>themisra.content-licensing/v1</code> Schema	9
4.1 Schema fields	9
4.2 Invariants	10
4.3 Why content-licensing depends on verifiable-content-provenance	10
5. Cross-Schema Composition	10
5.1 Reference shape	11
5.2 Cascade semantics	11
5.3 Multi-reference patterns	11
6. Royalty Distribution Mechanics	11
6.1 The fee split	11
6.2 Recommended <code>royalty_bps</code> calibration	12
6.3 Bridging to off-chain rails	12
7. Adversarial Considerations	13
7.1 License stripping	13
7.2 Derivative-work edge cases	13
7.3 Sublicensing	14
8. Roadmap	14
8.1 v1: <code>prompt-licensing</code> only	14
8.2 v1.5: <code>content-licensing</code> without provenance binding	14
8.3 v2: <code>content-licensing</code> with provenance binding	14
8.4 v2+: cross-chain royalty routing	14
9. Conclusion	14
References	15
Appendix A: Worked example: creator-economy round trip	15
Appendix B: Formal schema definitions	16

Abstract

Two new canonical schemas extend the Themisra Proof-of-Prompt receipt layer with licensing and royalty mechanics. `themisra.prompt-licensing/v1` registers a prompt with creator attribution and royalty terms; commercial reuse of the prompt pays the creator. `themisra.content-licensing/v1` registers an AI-generated artifact with creator attribution and license terms; commercial reuse of the content pays the creator.

The design choice is that **licensing lives as an application-layer schema, not as a chain-layer token primitive**. The chain stays minimal: a Proof-of-Prompt receipt is a clean claim, and the license is a separate attestation referencing that receipt. Cross-Schema Composition v0.2 §4.3 typed references bind licenses to their underlying receipts. Royalty distribution layers on top of per-schema-fees v0.2 §4.4 base-fee routing: each commercial use that references a licensed schema pays a fee split (recommended at v0.2: 25% burn, 35% attester, 30% creator, 10% builder).

The paper specifies the schema fields, the composition pattern, the royalty mechanics with concrete `royalty_bps` calibration ranges, and the adversarial considerations (license stripping, derivative-work edge cases, bounded-depth sublicensing). Appendix A traces a worked creator-economy round trip end-to-end, from initial prompt registration through derivative-work royalty payment.

1. Introduction

1.1 Why licensing belongs as a schema, not as a chain-layer token primitive

Proof-of-Prompt records that a prompt produced an output. That receipt is a *claim*, not a *license*. License terms vary across creators (some want commercial-only royalties, some allow derivative works with attribution, some are personal-use only), across jurisdictions (different copyright regimes apply to AI-generated artifacts differently), and across use cases (a one-off creative prompt has different licensing needs than a long-running creator-platform product). The underlying claim, by contrast, is invariant: the prompt-output linkage is the same regardless of how the creator wants to license it.

Layering license as a separate schema preserves the minimal-receipt invariant. A Proof-of-Prompt receipt remains a clean, single-purpose attestation; a separate licensing attestation references it and adds the terms. The two attestations can be revoked independently (license terms can change without invalidating the underlying receipt; the receipt can be revoked without affecting other licenses referencing it). This matches the Cross-Schema Composition v0.2 design philosophy: typed reference + cascade semantics rather than monolithic schemas.

The alternative would be a single fat Proof-of-Prompt schema with optional license fields. This is rejected for three reasons. First, it bloats the schema for users who do not need licensing (the majority of consumer Mneme usage). Second, it conflates two distinct claims (prompt-output linkage and license terms) under one attestation, making partial revocation impossible. Third, it would lock the chain into one license-vocabulary at the schema level, foreclosing alternative licensing patterns that might emerge.

1.2 Why now (or why v1.5)

Three forces converge in 2026:

- (a) **Per-schema-fees v0.2 ships with §4.4 base-fee routing.** The schema-author routing fraction `rho_sigma` is the protocol-level primitive that royalty distribution layers on top of. Without per-schema fee routing as a native primitive, royalty distribution would require application-layer plumbing (separate royalty contracts, off-chain accounting); with it, royalties become a direct extension of the existing fee-market mechanism.
- (b) **Cross-schema composition v0.2 §4.3 typed references** make schema-to-schema binding the canonical pattern. The license schemas declare typed inputs of `themisra.prompt-of-prompt/v1` (or `themisra.content-provenance/v1` when that ships); admission-time validation enforces the type constraint. Cascade semantics handle revocation across the licensing graph.
- (c) **The Themisra umbrella positioning (ligate-marketing#95)** gives canonical Themisra-side schemas beyond Proof-of-Prompt a defined design space. Licensing was identified as the natural second canonical schema family under the Themisra umbrella; this paper formalizes that design.

The note is **v1.5 territory**: licensing is a follow-on to the v1 mainnet receipt layer, not part of the day-1 surface. The v1 chain ships Proof-of-Prompt as the only Themisra-side canonical schema; v1.5 adds prompt-licensing; v2 adds content-licensing once verifiable-content-provenance ships. This phasing keeps the chain’s minimum-mainnet surface small.

1.3 The central question

Can the chain support per-prompt and per-content licensing with royalty distribution without expanding the chain-layer token primitive set, by composing existing primitives (Proof-of-Prompt receipts + typed references + per-schema fee routing) into a licensing-schema layer?

The paper’s answer is yes. The two licensing schemas (§3 and §4) carry the variable terms; CSC v0.2 §4.3 typed references bind them to underlying receipts (§5); per-schema-fees v0.2 §4.4 base-fee routing handles royalty distribution (§6); the result is creator-economy licensing as native chain functionality without adding new chain-layer primitives. No new token type, no new contract layer, no new application-layer plumbing.

1.4 Approach in brief

§2 surveys prior art (NFT-as-content patterns, ASCAP/BMI off-chain music licensing, Story Protocol IP-on-chain, EAS revocable attestations with license patterns). §3 specifies `themisra.prompt-licensing/v1` with eight fields plus invariants and version-bumping rules. §4 specifies `themisra.content-licensing/v1` mirroring §3’s structure with adaptations for artifact-level (vs prompt-level) attestation. §5 covers cross-schema composition: reference shape, cascade semantics, multi-reference patterns for derivatives. §6 covers royalty distribution mechanics: fee split (25% burn / 35% attester / 30% creator / 10% builder recommended), per-license-type `royalty_bps` calibration, off-chain rail bridging. §7 covers adversarial considerations: license stripping, derivative-work bright-line vs case-by-case, bounded-depth sublicensing with scope-monotonicity. §8 lays out the phased roadmap (v1 prompt-licensing → v1.5 content-licensing without provenance → v2 with provenance → v2+ cross-chain). §9 concludes. Appendix A walks a worked creator-economy round trip end to end.

1.5 Contributions

1. **Two schemas specified.** `themisra.prompt-licensing/v1` registers a prompt with creator attribution and royalty terms. `themisra.content-licensing/v1` registers an AI-generated artifact with creator attribution and licensing terms.
2. **Schema-layer-not-chain-layer argument.** Licensing variability lives in the schema; the chain stays minimal. The argument generalizes to other application-layer concerns.
3. **Cross-schema composition pattern.** Licensing schemas reference Proof-of-Prompt and content-provenance receipts via CSC v0.2 §4.3 typed references. Cascade semantics apply on receipt invalidation.
4. **Royalty distribution mechanics.** Each commercial use referencing a licensed schema pays a fee split: protocol burn + attester set + schema author + builder routing per per-schema-fees v0.2 §4.4. Concrete split percentages and rationale.
5. **Adversarial considerations.** License stripping (mitigated by content-provenance watermarking, where available). Derivative-work edge cases (bright-line vs case-by-case). Sublicensing (bounded depth, scope-monotonicity).

1.6 Scope and non-goals

In scope:

- Schema specifications for prompt and content licensing
- Cross-schema composition with existing receipt schemas
- Royalty distribution mechanics built on per-schema-fees v0.2 §4.4
- Adversarial model bounded by realistic threat surface

Explicitly out of scope:

- Off-chain royalty distribution to non-AVOW-denominated rails (out-of-scope; the chain settles in \$AVOW; fiat distribution is application-layer)
- Cross-chain license recognition (out-of-scope; lives in the cross-chain attestation portability follow-up paper, #136)
- Specifying the legal-enforceability framework (out-of-scope; smart contracts and chain attestations are not legal contracts; this paper specifies the technical mechanism, legal teams handle enforceability)
- ASCAP-style collective rights management (out-of-scope; could be built on top of these schemas by a separate organization, but is not part of the paper)

1.7 Document structure

§2 surveys prior art. §3 specifies `themisra.prompt-licensing/v1`. §4 specifies `themisra.content-licensing/v1`. §5 covers cross-schema composition. §6 specifies royalty distribution mechanics. §7 covers adversarial considerations. §8 lays out the roadmap. §9 concludes.

2. Background and Related Work

Four families of prior art inform the licensing-schema design.

2.1 NFT-as-content patterns

ERC-721 and ERC-1155 NFT patterns embed the content (or more commonly a hash pointer to off-chain storage) directly in the token. The token IS the content’s representation on chain; minting is the act of creation; transfer changes ownership; license terms ride in the token contract’s metadata or a separately referenced contract.

Themisra inverts this. The content stays off-chain (referenced by hash). The attestation is the receipt (a chain-state record that the prompt-output linkage exists). The license is a separate attestation referencing the receipt. Three downstream advantages:

- **Receipt is not a token.** No transfer semantics, no ownership transfer. The receipt is a historical record; ownership of derived rights is application-layer (license fields).
- **License terms can change without affecting receipt.** A creator who initially issued an unrestricted-use license can later issue a more restrictive one for new licensees without invalidating the underlying receipt.
- **License attestation has its own attestor set.** The creator signs licenses with their Mneme key; the receipt is signed with the user’s Mneme key at attest-time. The two can have different signers when authorship and licensing are separate (e.g., a publisher licensing on behalf of an author).

The token-as-content pattern carries license logic in the token contract; this paper’s design carries license logic in the schema layer. The schema-layer carries it cleanly; the contract layer carries it via call-out resolvers + auxiliary state.

2.2 ASCAP and BMI: off-chain music licensing

ASCAP (American Society of Composers, Authors and Publishers) and BMI (Broadcast Music, Inc.) are performing-rights organizations that collect royalties on behalf of music creators. They operate centralized infrastructure: membership lists, play-tracking via radio + streaming + venue reports, periodic royalty distributions based on play counts. Total combined annual collections are on the order of \$2-3B USD across the two organizations.

The model is well-understood at the operations level and a useful design analog. The paper’s chain-native analog: per-schema-fees \$4.4 base-fee routing replaces ASCAP’s distribution infrastructure with a protocol-level routing mechanism. Creators register schemas and receive routed fees automatically; no membership application, no central organization, no quarterly reconciliation cycle. The chain-native design loses ASCAP’s brand and existing licensee relationships; it gains automation, transparency, and disintermediation.

The intent is not to replace ASCAP for music licensing (which has decades of institutional momentum). The intent is to enable an analogous mechanism for AI-prompt + AI-content creator economies, where no comparable institutional infrastructure exists yet.

2.3 Story Protocol and other IP-on-chain projects

Story Protocol (story.foundation, launched 2024) ships IP-as-on-chain primitive on its own L1 chain. The design uses token-based IP-asset representations with attached license terms; smart contracts handle royalty distribution and derivative-work licensing. Other projects in the space (Tableland for relational on-chain data, Ledger for crypto-art licensing) ship similar designs at smaller scale.

Comparison to this paper:

- **Story Protocol:** token-and-contract-based, IP-asset is the central object, license terms attached to tokens, derivative works create new tokens. Higher protocol surface; richer expressiveness; chain-native to a chain specifically designed for IP.
- **This paper:** schema-and-attestation-based, attestation is the central object, license terms in separate attestations, derivative works create new attestations referencing the original via CSC. Lower protocol surface; composes with existing primitives; chain-agnostic in principle (the schemas can be ported to any chain with CSC + per-schema-fees equivalents).

Both are defensible design points. Story Protocol’s richness comes at the cost of locking the chain into a specific IP model; the paper’s compositional approach trades richness for protocol-surface economy. For Ligate Chain’s broader scope (AI receipts as the primary use case, with licensing as one layer above), the compositional approach fits better.

2.4 EAS revocable attestations + license patterns

EAS (Ethereum Attestation Service) supports revocable attestations natively (per [papers/eas-comparison/v0.2](#)). Emerging license patterns on EAS use schemas with fields like `royaltyBps`, `licenseType`, and `refUID` pointing to an underlying content credential.

Comparison to this paper:

- **EAS patterns:** untyped UID pointers; license-cascade-on-receipt-revocation must be tracked off-chain; royalty routing requires separate Solidity contracts intercepting attest events; no native economic floor for misbehaving attestors.
- **This paper:** typed CSC v0.2 §4.3 references; cascade fires automatically per CSC v0.2 §5; royalty routing uses native per-schema-fees v0.2 §4.4 `rho_sigma`; PoUA Lemma 1 floor underwrites attessor honesty.

The EAS pattern is operationally simpler to deploy today (no chain-specific primitives needed); the Ligate design is structurally tighter (less off-chain bookkeeping, more native composition). The cross-chain composition mechanism in [papers/cross-chain-portability/](#) could let Ligate-side license schemas reference EAS-side underlying credentials, combining the two.

3. The `themisra.prompt-licensing/v1` Schema

Specification of the prompt-licensing schema.

3.1 Schema fields

Eight fields, listed in canonical order:

Field	Type	Description
<code>prompt_id</code>	typed reference	CSC v0.2 §4.3 reference to a <code>themisra.proof-of-prompt/v1</code> attestation. Cascade fires per CSC v0.2 §5 if the underlying receipt is revoked.

Field	Type	Description
<code>creator_address</code>	chain address	The prompt author's chain address; must equal the Mneme-signing address on the referenced Proof-of-Prompt (proves licensor authority).
<code>royalty_bps</code>	uint16	Royalty rate in basis points (0 to 10000; 500 = 5%). Default recommendation table in §6.2.
<code>license_type</code>	enum	<code>personal-use</code> (0), <code>commercial-noncomm</code> (1), <code>commercial</code> (2), <code>unrestricted</code> (3). Sublicensing scope-monotonicity follows this partial order per §7.3.
<code>derivative_allowed</code>	bool	Whether derivative works are permitted (with the same license terms unless sublicensing is enabled).
<code>expiry_height</code>	uint64 (optional)	Block height beyond which the license is automatically invalid; 0 = no expiry.
<code>attestor_set_id</code>	uint64	Attestor set authorized to verify license-issuance and recall. Defaults to the Themisra umbrella set.
<code>payload_hash</code>	bytes32	Hash of an off-chain license-terms document (e.g., human-readable plain-English license text, jurisdictional disclaimers). Off-chain storage convention is the attestor set's responsibility.

The schema is intentionally narrow. License-specific terms beyond what the eight fields capture (e.g., territorial restrictions, specific exclusivity carve-outs) live in the off-chain `payload_hash` document; the chain-side schema does not interpret them. This keeps the schema's `crypto_suite` and admission-predicate surface bounded.

3.2 Invariants

Enforced at admission time by the chain runtime:

- **Reference validity:** `prompt_id` must resolve to a valid (non-revoked) Proof-of-Prompt at-

testation under `themisra.proof-of-prompt/v1`. If the referenced attestation does not exist or is revoked, the license attestation is rejected.

- **Authority match:** `creator_address` must equal the Mneme-signing address on the referenced Proof-of-Prompt. This prevents impersonation: a third party cannot license a prompt they did not author. The chain runtime checks this by hashing the referenced attestation's signature and comparing.
- **Rate bounds:** `royalty_bps` in `[0, 10000]`. Values outside this range are rejected as malformed.
- **Expiry sanity:** if `expiry_height > 0`, then `expiry_height > current_block_height` at issuance. Issuing an already-expired license is rejected as malformed.
- **Attestor set existence:** `attestor_set_id` must reference a registered attestor set at issuance time. Default value (Themisra umbrella set ID) auto-resolves if not specified.
- **Sublicensing scope:** when this attestation is a sublicense (referenced from a parent license attestation), the scope-monotonicity rule in §7.3 must hold. The chain runtime enforces this via the CSC v0.2 admission predicate.

3.3 Version-bumping rules

Schema version bumps follow CSC v0.2 §3.1 semver semantics. `v1` $\rightarrow$ `v2` within compatible range adds optional fields; major-version bump breaks composition contracts and requires consumer migration.

3.4 Worked example

See Appendix A for the worked end-to-end example.

4. The `themisra.content-licensing/v1` Schema

Specification of the content-licensing schema. Mirrors §3 structure but for AI-generated artifacts (images, audio, text outputs) rather than prompts.

4.1 Schema fields

Same eight-field structure as §3.1 with these adaptations for artifact-level (vs prompt-level) attestation:

Field	Type	Difference from §3.1
<code>artifact_hash</code>	<code>bytes32</code>	Replaces <code>prompt_id</code> . Hash of the AI-generated artifact (image, audio, document, etc.).

Field	Type	Difference from §3.1
<code>content_provenance_id</code>	typed reference (optional)	CSC v0.2 reference to a <code>themisra.content-provenance/v1</code> attestation (from <code>papers/verifiable-content-provenance/</code> , when v0.1+ ships). Allows verifiable artifact-identity binding.
<code>model_id</code>	string	Which AI model produced the artifact (Themisra metadata: e.g., <code>openai/dall-e-3</code> , <code>anthropic/claude-3-5-sonnet</code>).
<code>creator_address</code> , <code>royalty_bps</code> , <code>license_type</code> , <code>derivative_allowed</code> , <code>expiry_height</code> , <code>attestor_set_id</code> , <code>payload_hash</code>	as §3.1	Same semantics.

4.2 Invariants

Inherits all §3.2 invariants. Two additional invariants specific to content-licensing:

- **Artifact uniqueness:** `artifact_hash` must be unique across active `themisra.content-licensing/v1` attestations. Duplicate license attestations for the same artifact are rejected at admission, preventing competing licenses on the same content.
- **Optional provenance binding:** if `content_provenance_id` is set, it must reference a valid (non-revoked) content-provenance attestation under the schema named in `papers/verifiable-content-provenance/`. CSC v0.2 §5 cascade applies on its revocation: if the content-provenance attestation is invalidated, the dependent content-licensing attestation cascades to invalid.

4.3 Why content-licensing depends on verifiable-content-provenance

The optional `content_provenance_id` field becomes meaningful once verifiable-content-provenance v0.1 ships. Before then, content-licensing operates on `artifact_hash` alone; users supply the hash and trust that the licensor identified themselves correctly. After verifiable-content-provenance ships, the link is verifiable on chain.

5. Cross-Schema Composition

How the two licensing schemas compose with Proof-of-Prompt and content-provenance receipts using CSC v0.2 §4.3 typed references.

5.1 Reference shape

The `prompt_id` field in `themisra.prompt-licensing/v1` is a typed reference to a `themisra.proof-of-prompt/v1` attestation. The reference declares the expected input type; CSC v0.2 §4.3 admission-time validation checks the referenced attestation exists, type-matches, and satisfies any predicate (e.g., the proof-of-prompt’s signer must equal the licensing-schema’s `creator_address`).

5.2 Cascade semantics

If the underlying Proof-of-Prompt is revoked (e.g., the prompt author retroactively withdraws permission), CSC v0.2 §5 cascade invalidates dependent licensing attestations. Cascade is BFS-bounded; gas charged to the revocation root per CSC §5.5.

5.3 Multi-reference patterns

Some advanced patterns reference multiple receipts: a derivative-work license might reference both the original prompt-licensing AND the original Proof-of-Prompt. CSC v0.2 supports multi-input schemas; the paper specifies the recommended derivative-work reference set.

6. Royalty Distribution Mechanics

How royalties get paid when a commercial use references a licensed schema. Layered on per-schema-fees v0.2 §4.4 base-fee routing.

6.1 The fee split

Each commercial-use attestation referencing a licensed schema pays a per-schema base fee that splits four ways:

Recipient	Mechanism	v0.2 recommended share
Protocol burn sink	<code>tau_burn</code> from tokenomics v0.2 §7	25%
Attestor set (validators that included the attestation)	PoUA §6.1 validator-side share	35%
Schema author (the creator, via <code>rho_sigma</code>)	per-schema-fees v0.2 §4.4 routing	30%
Builder routing (sponsored-gas relayers like Iris)	per-schema-fees v0.2 §4.3	10%

The 25/35/30/10 split is a v0.2 starting recommendation. Governance can tune any of the four percentages within protocol-bounded ranges. The constraint `tau_burn` + creator-share + builder-share + validator-share = 100% must hold; the validator share is residual after the other three are set.

Rationale for the recommended split: 25% burn matches tokenomics v0.2 §7 steady-state recommendation; 30% creator share is meaningful enough to incentivize creators to register licenses (vs not bothering); 35% attester share keeps validator economics healthy; 10% builder share leaves room for sponsored-gas relayers (Iris and similar) to operate sustainably. The split is conservative on all four sides.

Per-schema variation: different licensing schemas can set different splits if governance approves. A prompt-licensing schema for high-volume creator-economy use might use a higher creator share (40%) and lower attester share (20%); a content-licensing schema for evidentiary use might invert this. The schema declares its preferred split at registration; governance approves.

6.2 Recommended royalty_bps calibration

Tabular recommendation by license type and use-case profile:

License type	Use case	Recommended <code>royalty_bps</code>
<code>personal-use</code> (0)	Individual non-monetized use	0
<code>commercial-noncomm</code> (1)	Educational, research, internal-org use	100-500 (1-5%)
<code>commercial</code> (2)	Direct monetization (resale, paid distribution)	500-2000 (5-20%)
<code>unrestricted</code> (3)	Creator explicitly waives	0

The ranges are calibrated against creator-economy benchmarks where available (Adobe Stock typical royalty range, ASCAP statutory rates, music streaming per-play rates inferred to bps equivalents). Specific high-value use cases (regulated-industry compliance attestations, large-language-model training corpus licensing) may justify rates outside these ranges; the chain does not cap at the protocol level beyond the `[0, 10000]` invariant.

Default if creator does not specify: `royalty_bps = 0` with `license_type = personal-use`. The creator must explicitly opt into a monetized license. This default is creator-protective: a license attestation that accidentally specifies commercial terms without intent is preferable to the inverse.

6.3 Bridging to off-chain rails

AVOW-denominated routed fees are received by the creator’s chain address at the moment of commercial-use attestation. The creator can hold, transfer, or spend the AVOW on chain natively. Bridging to fiat rails (USD, EUR, etc.) is application-layer: the creator routes received AVOW through an exchange or off-ramp (Wise, Mercury, or crypto-native services) to convert.

The paper specifies the chain-side flow only. Off-chain rail integration is a product concern, not a protocol concern. The creator economy enabled by this paper’s mechanism is AVOW-denominated by default; creators who want fiat distribution face the same friction as any other AVOW recipient. The native-delegation v0.2 Iris relayer infrastructure can in principle act as a fiat-bridge service for creator royalty distribution; the paper does not specify this but flags it as a v2+ Iris extension.

7. Adversarial Considerations

Three threat surfaces with mitigations.

7.1 License stripping

An attacker rewrites the artifact to remove the licensing reference, typically by stripping metadata or re-encoding the file. Three mitigation paths in increasing strength:

- **Honest cooperation:** most users want attribution. The mainstream creator economy operates on a cooperative model where attribution is socially expected (Reddit, Twitter, news organizations cite originators). License stripping is a minority adversarial concern at the volume level; the system does not need to defend against every possible adversarial action, only against the ones that would systematically undermine the creator economy.
- **Discovery via Atlas crawler** (`papers/verifiable-content-provenance/ §2.6 + ligate-marketing#96`). The Atlas verifier can flag artifacts whose content hash matches a licensed receipt but lacks the expected on-chain license attestation. Coverage depends on Atlas crawler reach; not 100% but materially deterrent.
- **Content-provenance watermarking** (depends on `papers/verifiable-content-provenance/ v0.1+ §4 + watermarking partner integrations`). If the artifact carries a cryptographic watermark identifying its license, stripping the metadata does not strip the watermark. Coverage depends on watermark-scheme robustness and watermark-bearing platform adoption.

Composition: honest cooperation handles the majority of cases; Atlas crawler handles the cooperative-but-careless tail; watermarking handles the adversarial tail. The three layers together provide bounded coverage; the §6.4 honest accounting acknowledges that 10-30% of adversarial usage remains undetected.

7.2 Derivative-work edge cases

When does a derivative work trigger royalty? Two design choices:

- **Bright-line rule:** explicit derivative declaration. The derivative-work creator references the original licensing schema in their own attestation (e.g., a derivative `themisra.content-licensing/v1` attestation has a typed reference to the parent license). The chain runtime enforces the royalty payment at admission. Pros: deterministic, auditable, low overhead. Cons: requires creator honesty in declaring derivatives (an undeclared derivative escapes royalty).
- **Case-by-case adjudication:** an attessor set (the Themisra umbrella set or a license-specific committee) determines whether a given attestation constitutes a derivative requiring royalty payment. Pros: covers undeclared derivatives. Cons: high operational overhead, slow turnaround, adjudication costs.

The paper recommends **bright-line (explicit declaration)** as the default with the Atlas verifier surfacing potential derivatives for community review. The bright-line rule is enforceable at the protocol layer; the Atlas verifier handles the discovery gap. For high-value content where undeclared derivatives carry material royalty losses, creators can opt into a case-by-case attessor-set committee for their specific license (configurable at registration); the protocol supports both modes via the `attessor_set_id` field's role in §3.1.

7.3 Sublicensing

A licensee further licenses to a third party. Bounded depth: max 2 sublicensing hops (configurable at registration). Scope-monotonicity: each sublicense has scope no broader than the parent (derived from the parent's `license_type` via a partial order).

8. Roadmap

Phased rollout aligned with the chain roadmap.

8.1 v1: prompt-licensing only

First ship `themisra.prompt-licensing/v1`. Simpler schema; clear use case (creator earns from prompt reuse); no content-provenance dependency. Targets H1 2027 chain v1.

8.2 v1.5: content-licensing without provenance binding

Ship `themisra.content-licensing/v1` with `content_provenance_id` as optional / unused. Users supply `artifact_hash` directly; license operates on hash equality alone.

8.3 v2: content-licensing with provenance binding

Once verifiable-content-provenance v0.1 ships, populate `content_provenance_id` and apply cascade semantics. Provides adversarially-robust artifact identity.

8.4 v2+: cross-chain royalty routing

Once the cross-chain attestation portability follow-up paper (#136) ships, royalty routing extends to off-chain creator addresses on Ethereum / Cosmos chains via Hyperlane-style bridging. Out of scope for this paper.

9. Conclusion

Two paragraphs. (1) Licensing belongs as a schema, not a chain-layer token primitive. The two schemas specified here (`themisra.prompt-licensing/v1` and `themisra.content-licensing/v1`) compose with existing receipt schemas via CSC v0.2 typed references. Royalty distribution rides on per-schema-fees v0.2 base-fee routing. No new chain primitive is introduced; the paper specifies how to combine existing primitives. (2) The roadmap is conservative: prompt-licensing first, content-licensing without provenance binding next, full content-provenance binding once that paper ships. The result is a layered creator-economy primitive that the chain supports natively without expanding its trust surface.

References

[v0.1: References to fill in at v0.2. Anchors:]

1. PoUA paper (this repo, papers/poua/), arXiv:2605.25844.
 2. Cross-Schema Composition paper (this repo, papers/cross-schema-composition/) §3.1, §4.3, §5.
 3. Per-Schema Fees paper (this repo, papers/per-schema-fees/) §4.4 (base-fee routing).
 4. Tokenomics paper (this repo, papers/tokenomics/) §7 (tau_burn calibration).
 5. Verifiable Content Provenance paper (this repo, papers/verifiable-content-provenance/) (planning).
 6. EAS (Ethereum Attestation Service) revocable-attestation documentation.
 7. Story Protocol whitepaper.
 8. ASCAP technical documentation on royalty distribution.
-

Appendix A: Worked example: creator-economy round trip

A worked end-to-end example showing creator-economy flow under the recommended `royalty_bps` = 500 (5%) and 25/35/30/10 fee split.

Setup. Alice is a prompt engineer who designs high-quality prompt templates for image generation. Bob is a content creator who wants to use Alice's prompt to produce derivative images for his commercial newsletter.

Step 1: Alice issues Proof-of-Prompt for prompt P. - Alice signs `themisra.proof-of-prompt/v1` with prompt hash `0xabcd...`, model identifier `openai/dall-e-3`, her chain address as attestor.
- Chain admits the attestation; UID `PoP_A` is assigned. - Alice pays the base fee for the proof-of-prompt schema (small, sub-cent USD equivalent in AVOW).

Step 2: Alice issues prompt-licensing attestation for PoP_A. - Alice signs `themisra.prompt-licensing/v1` with:
- `prompt_id` = `PoP_A` (typed CSC reference)
- `creator_address` = Alice's chain address
- `royalty_bps` = 500 (5%)
- `license_type` = `commercial` (2)
- `derivative_allowed` = `true`
- `expiry_height` = 0 (no expiry)
- `attestor_set_id` = Themisra umbrella set ID (default)
- `payload_hash` = hash of human-readable license text
- Admission predicate validates: `PoP_A` exists, Alice signed both (`creator_address` matches), `royalty_bps` in range, `expiry_height` valid. Attestation admitted.
- UID `License_A` is assigned. Alice pays the base fee for the prompt-licensing schema.

Step 3: Bob uses prompt P to produce derivative D. - Bob runs DALL-E 3 with prompt P, producing image D with hash `0xeffd...`
- Bob signs `themisra.proof-of-prompt/v1` with derivative declaration: prompt hash `0xabcd...` (Alice's), output hash `0xeffd...` (Bob's), `parent_license_id` = `License_A` (typed reference).
- Admission predicate validates: prompt hash matches `PoP_A`, parent license `License_A` exists and is non-revoked, `derivative_allowed` = `true` on parent license.
- Attestation `PoP_B` admitted. Bob pays the base fee.

Step 4: Bob commercially uses derivative D (publishes in newsletter). - Bob signs a `themisra.content-licensing/v1` commercial-use attestation referencing `PoP_B` as the underlying receipt and `License_A` as the upstream license.
- The base fee for this commercial-use attestation, in the per-schema-fees v0.2 §4.4 framework, includes the prompt-license royalty as a routed component.

Step 5: Fee split execution. Suppose Bob’s commercial-use attestation pays 100 AVOW base fee (illustrative). The chain runtime splits: - 25% burn = 25 AVOW to protocol sink - 35% attestor = 35 AVOW to the validators that included Bob’s attestation - 30% creator = 30 AVOW to Alice’s chain address (per `rho_sigma`, with the upstream-license-creator routing taking precedence over the schema-author routing) - 10% builder = 10 AVOW to any sponsored-gas relay (e.g., if Bob used Iris to publish, the routed share covers Iris’s gas costs)

Plus, the 5% royalty per `License_A`: Bob also pays a separate 5 AVOW royalty (5% of 100) to Alice through the per-schema-fees routing mechanism. The royalty is in addition to the standard fee split.

Net result. Alice receives 30 AVOW (creator share of base fee) + 5 AVOW (royalty) = 35 AVOW per commercial use. Over the lifetime of Alice’s prompt, accumulated royalties scale with derivative-work volume. The chain runtime handles the routing; no application-layer plumbing needed.

Chain-state trace. Each of Steps 1-4 produces one chain attestation; Step 5 produces fee-routing chain-state updates (no separate attestation). Total: 4 attestations + several fee-routing state updates. Cumulative storage cost for the round trip: approximately 1 KB attestation bodies + amortized state-tree updates. Gas cost: bounded by per-schema base fees; for low-volume schemas at v0 parameters this is well under \$1 USD equivalent.

Appendix B: Formal schema definitions

[**v0.1:** At v0.2: full schema definitions in CSC v0.2 §3.1 format. Input-type sets, predicate functions, version-bumping rules, cascade behavior.]